Introduction to Neural Networks

MY459 Computational Text Analysis and Large Language Models

https://lse-my459.github.io/

Friedrich Geiecke

Outline

- Introduction
- Fundamental architectures
- Training
- Coding

This lecture

- In this lecture, we will discuss basics of artificial neural networks and deep learning
- The subsequent lectures on embeddings, transformers, and language models will build on these topics

Let's begin with some history

- McCulloch and Pitts (1943) created a computational model for neural networks
- Research on the topic continued throughout the 20th century, however, at times as a niche field
- In the early 21st century, neural networks began outperforming other models at scale
- In particular, AlexNet (Krizhevsky et al., 2012) won the 2012 ImageNet (an image classification) competition by a large margin which was unexpected for many researchers at the time

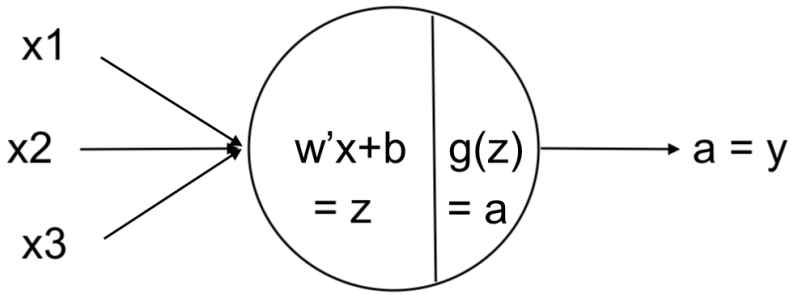
Why have artificial neural networks made so much progress in recent years?

- Data: Much more (labeled) data available
- Hardware: Much more compute available (Moore's law); extensive use of GPUs for parallelisation of tensor operations
- Software: Improved architectures, libraries, optimisation algorithms, etc.

Outline

- Introduction
- Fundamental architectures
- Training
- Coding

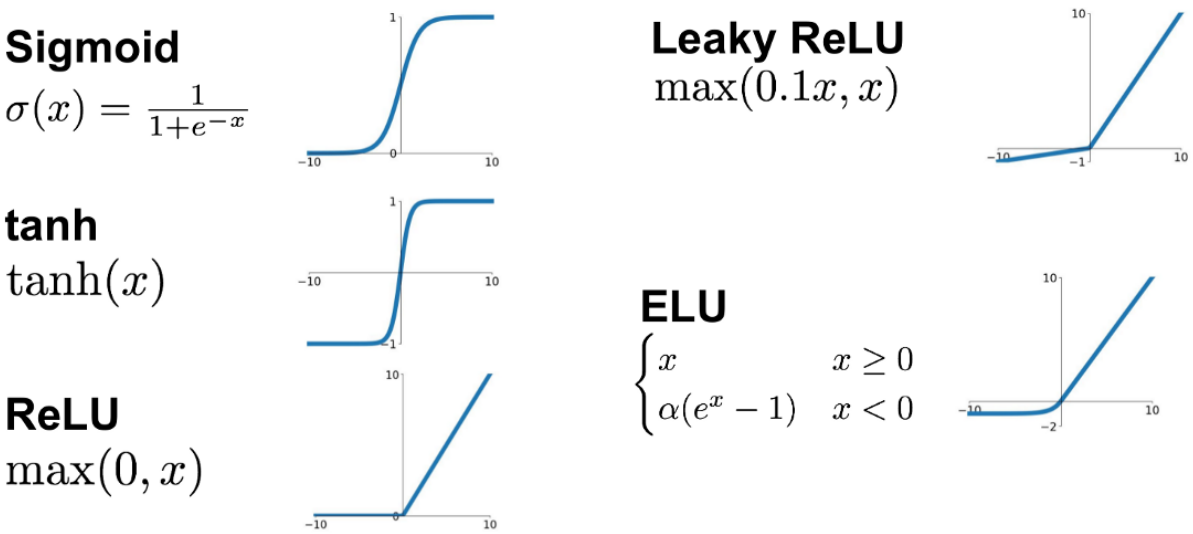
Simplest architecture: Single layer perceptron



- **Single layer perceptron (SLP)** here: A (**feedforward**) neural network with no hidden layer and an output layer with a single neuron and activation
- $z = x'w + b = x_1 w_1 + x_2 w_2 + x_3 w_3 + b$ with 3-dimensional **weights** vector $w = (w_1, w_2, w_3)$ and **bias** b
- $a = g(z)$ where $g(z)$ is called **activation function**
- Circles in figures commonly depict both z and a values
- Exercise: With **sigmoid activation** $g(z) = \frac{1}{1+e^{-z}}$, this is the same as a ...

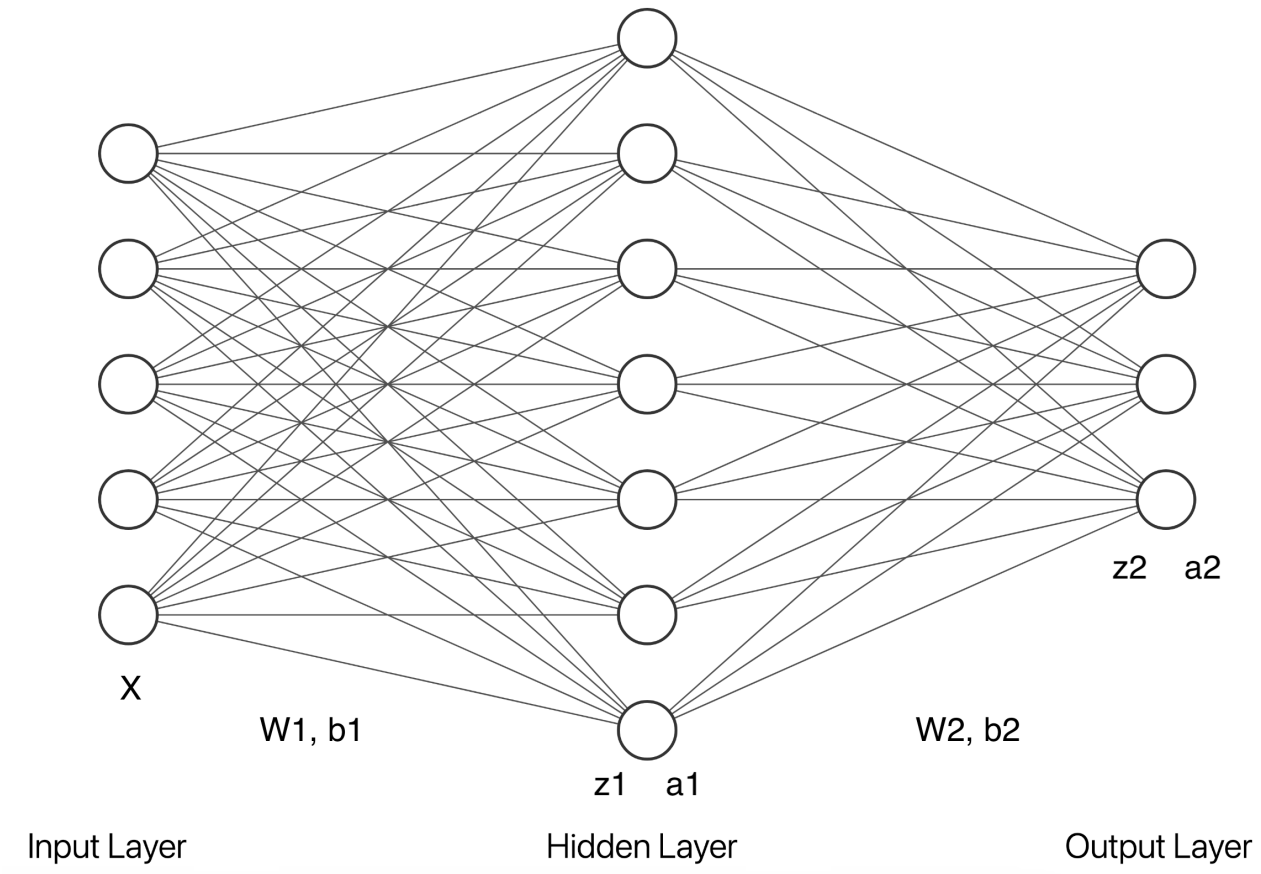
Activation functions

- Activation functions add non-linearity
- Sigmoid can have close to zero gradients for both negative and positive values and imply slow learning
- ReLU (rectified linear unit) and variants are commonly used today



Some common activation functions. Image from Jadon (2018)

Classification MLP with one hidden layer



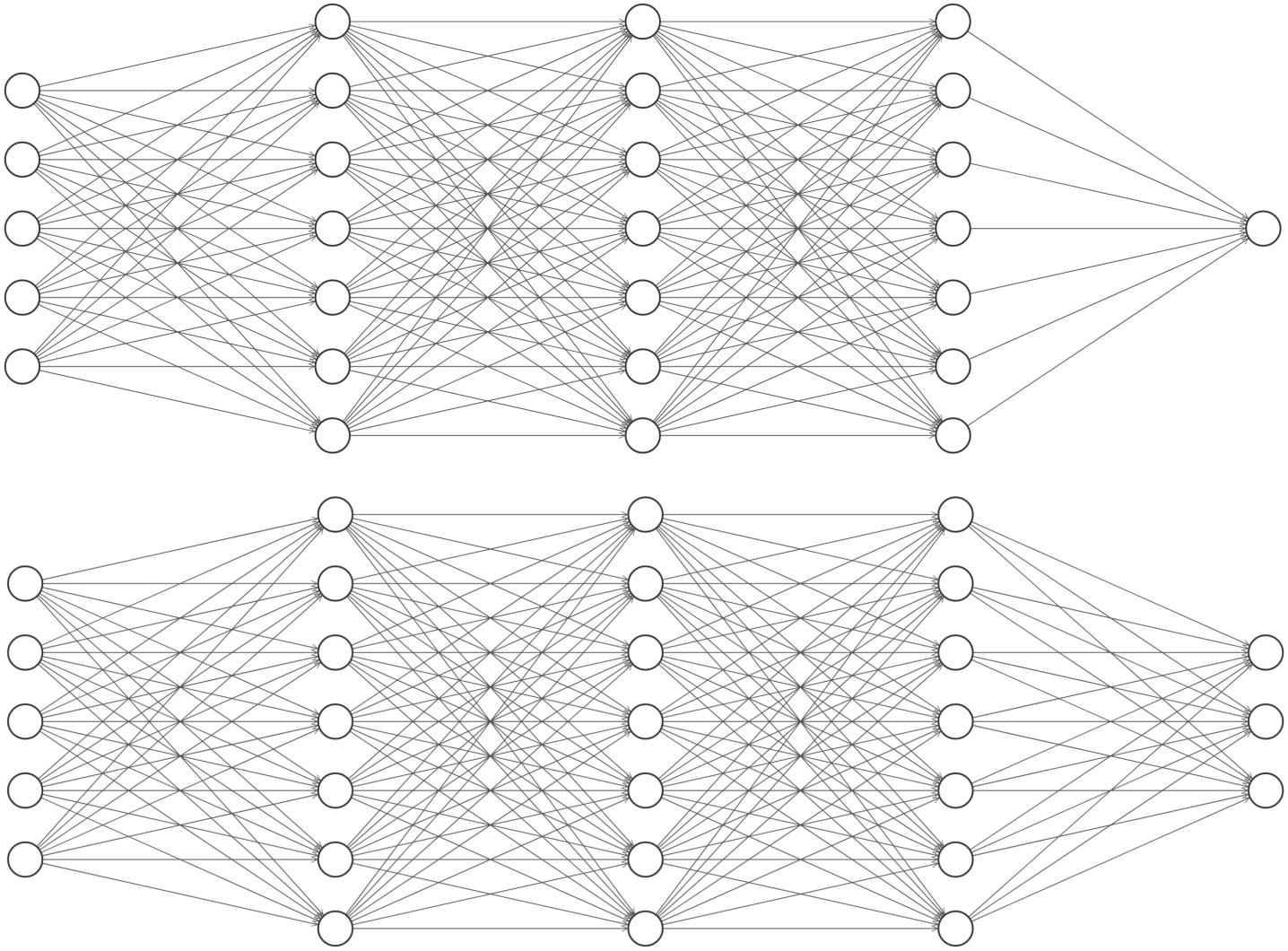
Drawn with `http://alexlenail.me/NN-SVG/index.html`

In more detail

Forward pass expressed in one line: $x \underbrace{\rightarrow}_{\cdot W^{(1)},\ +b^{(1)}} z^{(1)} \underbrace{\rightarrow}_{g(\cdot)} a^{(1)} \underbrace{\rightarrow}_{\cdot W^{(2)},\ +b^{(2)}} z^{(2)} \underbrace{\rightarrow}_{s(\cdot)} a^{(2)} = p$

- Input layer: $\underbrace{x}_{5\times 1}$
- Hidden layer z: $\underbrace{W^{(1)}}_{7\times 5} \underbrace{x}_{5\times 1} + \underbrace{b^{(1)}}_{7\times 1} = \underbrace{z^{(1)}}_{7\times 1}$
- Hidden layer activation: $\underbrace{a^{(1)}}_{7\times 1} = \underbrace{g(z^{(1)})}_{7\times 1}$ (e.g. ReLU applied element-wise with $g(z_i) = max\{0, z_i\}$)
- Output layer z: $\underbrace{W^{(2)}}_{3\times 7} \underbrace{a^{(1)}}_{7\times 1} + \underbrace{b^{(2)}}_{3\times 1} = \underbrace{z^{(2)}}_{3\times 1}$
- Output activation: $\underbrace{s(z^{(2)})}_{3\times 1} = \underbrace{a^{(2)}}_{3\times 1} = \underbrace{p}_{3\times 1}$ (softmax function applied element-wise $s(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}$)

Exemplary MLPs with more hidden layers



Drawn with `http://alexlenail.me/NN-SVG/index.html`

Regression and classification

- Neural networks are used for both regression and classification
- Most commonly for regression, the output activation is linear, i.e., $g(z) = z$
- For two-class classification, it may be sigmoid, i.e., $g(z) = \frac{1}{1+e^{-z}}$
- For multi-class classification, its generalisation is used, the softmax: $s(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}$

Discussion

- Conceptually, the neural networks discussed here are simply function approximators that fit the functional relationship between inputs X and outputs y, just like other regression and classification models
- Yet, adding further neurons, hidden layers, etc. allows neural networks to fit highly complex functions
- Activations are key as well, stacking only linear layers could only create a linear model

Universal approximation theorem

- Under some regularity conditions already a single hidden layer neural network can approximate continuous functions, that map from $[0, 1]^K$ to the real number line, arbitrarily closely (see Cybenko, 1989; Hornik et al., 1989; Hornik, 1991, for further details)
- Note that this says nothing about how we can practically find the optimal weights of such a network, just that it exists
- Empirically, networks with many hidden layer tend to find the better solutions

Outline

- Introduction
- Fundamental architectures
- Training
- Coding

Training

- So far we can randomly initialise the weights and biases of the network and compute the forward pass to make (not very good) predictions
- How do we learn better values for the weights and biases in order to improve predictions? We can minimise a loss function via gradient descent
- Commonly this loss function is mean squared error for regression or cross-entropy for classification

Loss functions

- Mean squared error loss for one observation:

$$L(y_i, p_i) = (y_i - p_i)^2$$

- Cross-entropy loss for one observation:

$$L(y_i, p_i) = -\sum_{k=1}^{K} y_{i,k} \log(p_{i,k})$$

Loss functions

- Summarise all weights, $W^{(1)}, W^{(2)}, \ldots$ and biases $b^{(1)}, b^{(2)}, \ldots$ in one vector θ
- For full training dataset:

$$J(\theta; \{x_i, y_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, p(x_i, \theta))$$

- To continue with gradient descent, we will need to compute the gradient of this loss function
- Recall the chain rule: If $f(g(x))$, then $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial x}$

Gradient of loss

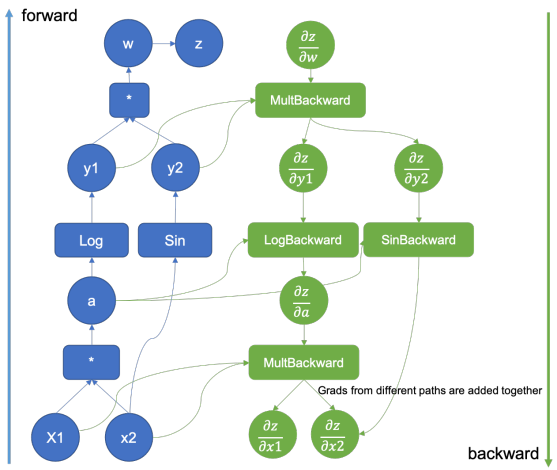
- Recall that the path through the network in the forward pass has the form:

$$x \underbrace{\rightarrow}_{W^{(1)},b^{(1)}} z^{(1)} \underbrace{\rightarrow}_{g(\cdot)} a^{(1)} \underbrace{\rightarrow}_{W^{(2)},b^{(2)}} z^{(2)} \underbrace{\rightarrow}_{s(\cdot)} a^{(2)} = p$$

- Thus, applying the chain rule we get the gradient (assume scalars for simplicity here):
  - $\frac{\partial J(\theta)}{\partial W^{(2)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial W^{(2)}}$
  - $\frac{\partial J(\theta)}{\partial b^{(2)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial b^{(2)}}$
  - $\frac{\partial J(\theta)}{\partial W^{(1)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial W^{(1)}}$
  - $\frac{\partial J(\theta)}{\partial b^{(1)}} = \frac{\partial J(\theta)}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial b^{(1)}}$
- Combine/stack to get gradient $\frac{\partial J(\theta)}{\partial \theta}$ or also often denoted $\nabla_\theta J(\theta)$

Computation graphs

- Modern libraries such as PyTorch and TensorFlow process networks as computation graphs
- The gradients they compute flow through the individual elements of that graph and allow a wide range of architectures
- For details, see e.g. MY475



Example from PyTorch blog

Gradient descent

- Computing the gradient for the entire dataset (full batch gradient descent) is usually computationally too costly
- Instead, we typically approximate the gradient of the full training data with a small batch of data (mini batch gradient descent)
- Importantly, since loss functions for neural networks are typically non-convex, these algorithms are not guaranteed to converge to global optima. In practice, saddle points and flat regions are common obstacles

Mini-batch gradient descent

- Randomly initialise weights; select a learning rate α. Then repeat the following for E epochs or until approximate convergence:

  1. Shuffle all observations in the training dataset and divide into $\left\lceil \frac{n}{B} \right\rceil$ batches
  2. For each batch with observations:
     - Update $\theta \leftarrow \theta - \alpha \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta L\left(y_i, p(x_i, \theta)\right)$

- Commonly used current optimisers such as Adam additionally add features like momentum

Regularisation

- With their very high numbers of parameters, neural networks can memorise large amounts of data and over-fit relatively easily
- Possible approaches to counter over-fitting are:
  - Add L1 ($\sum |\theta_i|$) or L2 ($\sum \theta_i^2$) norms of parameters to the loss function
  - Dropout: Randomly set a fraction of p neurons in a layer to zero during training and thereby force varying sets of neurons to learn patterns. This can be conceptualised as training an ensemble
  - Stop the gradient descent once the loss on some validation set increases
  - Add noise to activities during training

To improve training

- Tune various hyperparameters (hidden layers, activations, neurons per hidden layer, etc.)
- Batch/layer norm: Normalisation over observations in a batch or activations in a layer
- Residual connections/skip connections: Skipping some layers and allowing information to flow more easily
- Often helpful to first fit the model to a very small sample of the data and see whether it can fit it perfectly. If not, there might be some issues with the architecture

Outline

- Introduction
- Fundamental architectures
- Training
- Coding

Coding

- To conclude, we will train a simple MLP network for text classification with PyTorch and then replicate the outcome from scratch
- 03-mlp-text-classification-pytorch.ipynb
- 04-mlp-text-classification-from-scratch.ipynb

Materials for further study

- Visualisations of neural networks and gradient descent on the Youtube channel *3Blue1Brown*: `https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi`

- Extensive course on neural networks for text (Karpathy, 2023): `https://www.youtube.com/watch?v=VMj-3S1tku0&list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ`

References I

Cybenko, George, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, 1989, *2* (4), 303–314.

Hornik, Kurt, "Approximation capabilities of multilayer feedforward networks," *Neural networks*, 1991, *4* (2), 251–257.

__, Maxwell Stinchcombe, and Halbert White, "Multilayer feedforward networks are universal approximators," *Neural networks*, 1989, *2* (5), 359–366.

Karpathy, Andrej, "Lecture: Intro to Large Language Models," 2023. `https://www.youtube.com/watch?v=zjkBMFhNj_g`.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, 2012, *25.*

McCulloch, Warren S and Walter Pitts, "A logical calculus of the ideas immanent in nervous activity," *The bulletin of mathematical biophysics*, 1943, *5*, 115–133.